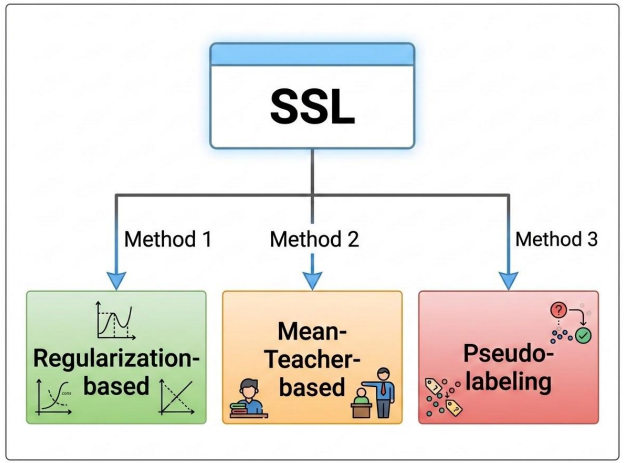
# Reinforcement Learning Guided Semi-Supervised Learning - A review

Bucur Denisa - Group 507
Tănase Victor-Flavian - Group 507
Olăeriu Vlad-Mihai - Group 507
Adam Emanuel - Group 511

# Introduction

# Semi-supervised Learning(SSL)

- Bridges the gap between supervised and unsupervised learning
- Makes the most of scarce labeled data and abundant unlabeled data



# Problem Statement

# The limitations of SSL

- Still a **significant challenge** to achieve **high performance on unlabeled data**
- **Highly reliant on heuristics**: predefined rules or heuristics to generate “pseudo-labels”
- **Constraint to Standard Loss Functions**: limited to differentiable loss functions and regularization techniques
- **Overfitting** : overfitting and difficulty creating accurate reliable pseudo-labels.
- **Complexity**: designing and coordinating multiple loss functions makes the problem complex

# Related Work

## Regularization

Small input perturbations

Virtual Adversarial Training(VAT)

## Teacher-Student

Student Network mimics a teacher network

Temporal ensembling for training stability

Mean Teacher

Interpolation Consistency Training(ICT)

## Pseudo-labeling

Generates labels using the model trained on the labeled data

Selection is made based on confidence

Mix Match Frameworks

# SSL as a One Armed Bandit Problem

- State = labeled and unlabeled data
- Agent(Policy Function) = the prediction model
- Action = generation of pseudo-labels(probabilities)
- Reward = how well does the model generalize

# Proposed Method

# Problem formulation

- Small sample of labeled data: $\mathcal{D}^l = (X^l, Y^l) = \{(x_i^l, \mathbf{y}_i^l)\}_{i=1}^{N^l}$
- Large number of unlabeled data: $\mathcal{D}^u = X^u = \{x_i^u\}_{i=1}^{N^u}$, with $N^u \gg N^l$
- The goal is to train a C-classifier that generalizes well on test data: $f_\theta : \mathcal{X} \to \mathcal{Y}$

# The One-Armed Bandit Problem

- Single Step Markov Process: **one action, one reward**
- Objective: learn an objective function that maximizes the one time reward
- The actions of the agent don’t affect the environment

# SSL as a RL problem

- One-armed bandit problem - single action -> reward
- The C-classifier - the learning policy
- Objective: learn an optimal policy $\pi^*$ that maximizes the reward for a given state **s**
  - Formally: $\pi^\star = \arg\max_\pi \ J_r(\pi) = \sum_a \pi(a|s)\mathcal{R}(s,a).$
- The state is defined as: $s = (X^l, Y^l, X^u)$
- Policy function: classifier defined by $\theta$: $\pi_\theta$
- Action: the probability vector produced by $\pi$ : $\pi_\theta(\cdot|s)$

# Reward function

- Reward function: the feedback to evaluate the performance of the action
- Data mixup: produce new data points from labeled and pseudo labeled data
- Strategy:
  - replicate the labeled dataset $\mathcal{D}^l$ by a factor of $r = \lceil \frac{N^u}{N^l} \rceil$ times
  - Shuffle the data and mix labeled and pseudo labeled data:

$$x_i^{\mathrm{m}} = \mu\, x_i^u + (1-\mu)\, x_i^l, \qquad \mathbf{y}_i^{\mathrm{m}} = \mu\, \mathbf{y}_i^u + (1-\mu)\, \mathbf{y}_i^l$$ where μ~Beta(1,1)

- Reward function: negative MSE between the prediction of the mixup point and the mixup label

$$\mathcal{R}(s, a; \mathrm{sg}[\theta]) = \mathcal{R}(X^l, Y^l, X^u, Y^u; \mathrm{sg}[\theta]) = -\frac{1}{C \cdot N^{\mathrm{m}}} \sum_{i=1}^{N^{\mathrm{m}}} ||P_\theta(x_i^{\mathrm{m}}) - \mathbf{y}_i^{\mathrm{m}}||_2^2$$

# RL Loss

- The Setup: The reward function is non-differentiable - we cannot directly backpropagate through it
- Proposed Solution: RL loss function that maximizes the reward while forcing the network to make confident predictions
- The Mechanism (KL-Divergence Weighted Reward):
  - Calculates the KL-Divergence between the network's prediction and a uniform distribution .
  - Multiplies this measured distance by the received reward .
- Maximizing the distance from a uniform distribution lowers the entropy of the prediction

$$\mathcal{L}_{\text{rl}}(\theta) = -\mathbb{E}_{\mathbf{y}_i^u \sim \pi_\theta} \mathbf{KL}(\mathbf{e}, \mathbf{y}_i^u) \mathcal{R}(s, a; \text{sg}[\theta]) = -\mathbb{E}_{x_i^u \in \mathcal{D}_u} \mathbf{KL}(\mathbf{e}, P_\theta(x_i^u)) \mathcal{R}(s, a; \text{sg}[\theta])$$

# Teacher-Student framework

- Framework is widely used in SSL for learning stability
- Start with 2 identical networks
- The Student (S): Learns actively via standard gradient descent
- The Teacher(T): Never trains via gradient descent. Uses Exponential Moving Average(EMA)

$$\theta_T = \beta\,\theta_T + (1-\beta)\,\theta_S$$

- $\beta$ - the decay rate, usually 0.999
- Teacher is used to generate the Actions, while the Student is the one tested on the MixUp data and updated

# Final Combined Loss

The RLGSSL framework optimizes the student network by combining reinforcement learning with standard semi-supervised signals through a joint objective function:

- **The Joint Loss Formula**: $\mathcal{L}(\theta_S) = \mathcal{L}_{\text{rl}} + \lambda_1 \mathcal{L}_{\text{sup}} + \lambda_2 \mathcal{L}_{\text{cons}}$
  - ***Supervised Loss***: Standard cross-entropy on available labeled data

$$\mathcal{L}_{\text{sup}}(\theta_S) = \mathbb{E}_{(x^l, \mathbf{y}^l) \in \mathcal{D}^l} \left[ \ell_{CE} \left( P_{\theta_S}(x^l), \mathbf{y}^l \right) \right]$$

  - ***Consistency Loss***: For prediction alignment between student and teacher on unlabeled data

$$\mathcal{L}_{\text{cons}}(\theta_S) = \mathbb{E}_{x^u \in \mathcal{D}^u} \left[ \text{KL} \left( P_{\theta_S}(x^u), P_{\theta_T}(x^u) \right) \right]$$

**Algorithm 1** Pseudo-Label Based Policy Gradient Descent

**Input:** $\mathcal{D}^l, \mathcal{D}^u$, and extended $\widetilde{\mathcal{D}}^l$; initialized $\theta_S, \theta_T$; hyperparameters
**for** $iteration = 1$ to maxiters **do**
  **for** $x_i^u \in \mathcal{D}^u$ **do**
    Compute soft pseudo-label vector $\mathbf{y}_i^u = P_{\theta_T}(x_i^u)$ to form $(x_i^u, \mathbf{y}_i^u)$
  **end for**
  Generate mixup data $\mathcal{D}^m = (X^m, Y^m)$ on $\mathcal{D}^u$ and $\widetilde{\mathcal{D}}^l$ using Eq.(1) with shuffling
  **for** $step = 1$ to maxsteps **do**
    Draw a batch of data $B = \{(x_i^m, \mathbf{y}_i^m)\}$ from $\mathcal{D}^m$
    Calculate the reward function $\mathcal{R}(\cdot; sg[\theta_S])$ using the batch $B$
    Compute the objective in Eq.(7)
    Update the policy parameters $\theta_S$ via gradient descent
    Update teacher model $\theta_T$ via EMA in Eq.(4)
  **end for**
**end for**

# Experiments

# Datasets

**CIFAR-10:**

- Experiments conducted with **250, 1000, 2000, and 4000** labeled samples.

**CIFAR-100:**

- Evaluated with **2500, 4000, and 10000** labeled samples.

**SVHN (Street View House Numbers):**

- Tested with **500 and 1000** labeled samples.

**STL-10:**

- Standard evaluation using **1000** labeled images.

# Computer Vision Architectures

**Multi-Architecture Validation:**

- The framework was tested across various architectures to demonstrate model-agnostic effectiveness.

**CNN-13:**

- A standard 13-layer Convolutional Neural Network used as the primary baseline for architectural comparison.

**Wide Residual Networks (WRN):**

- **WRN-28-2:** Used for CIFAR-10 and SVHN (28 layers deep, expansion factor of 2).
- **WRN-28-8:** A wider variant used for the more complex **CIFAR-100** dataset to capture richer features.
- **WRN-37-2:** Specifically deployed for the **STL-10** dataset to handle its higher resolution and unique data distribution.

# Evolution of SSL Algorithms

**Phase 1: Decision Boundary Stability**

- *Models:* Mean Teacher, VAT.
- *Concept:* **Consistency Regularization.** Focused on smoothing the model's output against input noise/perturbations to ensure stable predictions.

**Phase 2: Latent Space Interpolation**

- *Models:* MixMatch, ReMixMatch, ICT.
- *Concept:* **MixUp Data Augmentation.** Using linear combinations of samples to regularize the model and improve generalization in the space between classes.

**Phase 3: Static Heuristics**

- *Models:* FixMatch, MarginMatch.
- *Concept:* **Confidence-based Thresholding.** Converting high-confidence predictions into "hard labels" using fixed, human-defined thresholds.

**Phase 4: Adaptive Optimization**

- *Models:* Meta Pseudo-Labels (MPL), **RLGSSL**
- *Concept:* **Reinforcement Learning Policies.** Moving from fixed rules to an agent-based approach. The model learns a dynamic labeling policy optimized via reward signals.

# Comparison

| Dataset | CIFAR-10 | | | CIFAR-100 | |
|---|---|---|---|---|---|
| Number of Labeled Samples | 1000 | 2000 | 4000 | 4000 | 10000 |
| Supervised | $39.95_{(0.75)}$ | $27.67_{(0.12)}$ | $20.42_{(0.21)}$ | $58.31_{(0.89)}$ | $44.56_{(0.30)}$ |
| Supervised + MixUp [39] | $31.83_{(0.65)}$ | $24.22_{(0.15)}$ | $17.37_{(0.35)}$ | $54.87_{(0.07)}$ | $40.97_{(0.47)}$ |
| Π-model [6] | $28.74_{(0.48)}$ | $17.57_{(0.44)}$ | $12.36_{(0.17)}$ | $55.39_{(0.55)}$ | $38.06_{(0.37)}$ |
| Temp-ensemble [6] | $25.15_{(1.46)}$ | $15.78_{(0.44)}$ | $11.90_{(0.25)}$ | - | $38.65_{(0.51)}$ |
| Mean Teacher[8] | $21.55_{(0.53)}$ | $15.73_{(0.31)}$ | $12.31_{(0.28)}$ | $45.36_{(0.49)}$ | $35.96_{(0.77)}$ |
| VAT [5] | $18.12_{(0.82)}$ | $13.93_{(0.33)}$ | $11.10_{(0.24)}$ | - | - |
| SNTG [14] | $18.41_{(0.52)}$ | $13.64_{(0.32)}$ | $10.93_{(0.14)}$ | - | $37.97_{(0.29)}$ |
| Learning to Reweight [40] | $11.74_{(0.12)}$ | - | $9.44_{(0.17)}$ | $46.62_{(0.29)}$ | $37.31_{(0.47)}$ |
| MT + Fast SWA [13] | $15.58_{(0.12)}$ | $11.02_{(0.23)}$ | $9.05_{(0.21)}$ | - | $33.62_{(0.54)}$ |
| ICT [15] | $12.44_{(0.57)}$ | $8.69_{(0.15)}$ | $7.18_{(0.24)}$ | $40.07_{(0.38)}$ | $32.24_{(0.16)}$ |
| RLGSSL (Ours) | $\mathbf{9.15}_{(0.57)}$ | $\mathbf{6.90}_{(0.11)}$ | $\mathbf{6.11}_{(0.10)}$ | $\mathbf{36.92}_{(0.45)}$ | $\mathbf{29.12}_{(0.20)}$ |

Using a CNN-13 backbone, RLGSSL achieved a 9.15% average test error on CIFAR-10 (1k labels), outperforming *Learning to Reweight* by 2.59%. Based on average errors and standard deviations, it maintained its lead at 2k and 4k labels, beating the second-best method (ICT) by margins of 1.79% and 1.07% .

| Dataset | CIFAR-10 | | | CIFAR-100 | | SVHN |
|---|---|---|---|---|---|---|
| Number of Labeled Samples | 250 | 1000 | 4000 | 2500 | 10000 | 1000 |
| Mean Teacher [8] | $32.32_{(2.30)}$ | $17.32_{(4.00)}$ | $10.36_{(0.25)}$ | $53.91_{(0.57)}$ | $35.83_{(0.24)}$ | $5.65_{(0.45)}$ |
| ICT [15] | - | - | $7.66_{(0.17)}$ | - | - | $3.53_{(0.07)}$ |
| MixMatch [1] | $11.05_{(0.15)}$ | $7.75_{(0.32)}$ | $6.24_{(0.06)}$ | $39.94_{(0.37)}$ | $28.31_{(0.33)}$ | $3.27_{(0.31)}$ |
| ReMixMatch [17] | $5.44_{(0.05)}$ | $6.27_{(0.34)}$ | $6.24_{(0.06)}$ | $27.14_{(0.23)}$ | $23.78_{(0.12)}$ | $3.27_{(0.31)}$ |
| FixMatch [19] | $5.07_{(0.35)}$ | - | $4.26_{(0.05)}$ | $28.29_{(0.11)}$ | $22.60_{(0.12)}$ | $2.28_{(0.11)}$ |
| Meta-Semi [41] | - | $7.34_{(0.22)}$ | $6.10_{(0.10)}$ | - | - | - |
| Meta Pseudo-Labels [25] | - | - | $3.89_{(0.07)}$ | - | - | $1.99_{(0.07)}$ |
| MarginMatch [42] | $\mathbf{4.73}_{(0.12)}$ | - | $3.98_{(0.02)}$ | $23.71_{(0.13)}$ | $21.39_{(0.12)}$ | $1.93_{(0.01)}$ |
| RLGSSL (Ours) | $5.01_{(0.27)}$ | $\mathbf{4.92}_{(0.25)}$ | $\mathbf{3.52}_{(0.06)}$ | $\mathbf{23.18}_{(0.43)}$ | $\mathbf{20.15}_{(0.34)}$ | $\mathbf{1.92}_{(0.05)}$ |

Using WRN-28-2 (CIFAR-10/SVHN) and WRN-28-8 (CIFAR-100) backbones, RLGSSL demonstrated superior robustness across all datasets, specifically improving over ReMixMatch by 1.35% on CIFAR-10 (1k labels) and outperforming Meta Pseudo-Labels at 4k labels, while also surpassing the state-of-the-art MarginMatch on both CIFAR-100 and SVHN.

| | VAT [5] | Π-model [6] | Temp-ensemble [6] | MT [8] | ICT [15] | SNTG [14] | RLGSSL (Ours) |
|---|---|---|---|---|---|---|---|
| SVHN/500 | - | $6.65_{(0.53)}$ | $5.12_{(0.13)}$ | $4.18_{(0.27)}$ | $4.23_{(0.15)}$ | $3.99_{(0.24)}$ | $\mathbf{3.12}_{(0.07)}$ |
| SVHN/1000 | $5.42_{(0.00)}$ | $4.82_{(0.17)}$ | $4.42_{(0.16)}$ | $3.95_{(0.19)}$ | $3.89_{(0.04)}$ | $3.86_{(0.27)}$ | $\mathbf{3.05}_{(0.04)}$ |

On the SVHN dataset using a CNN-13 backbone, RLGSSL outperformed all SSL techniques by achieving the lowest average test errors of 3.12% (500 labels) and 3.05% (1k labels), surpassing the second-best method (*SNTG*) by margins of 0.87% and 0.81%, respectively, based on 5-trial averages and standard deviations.

# Ablation Study

# Setup

**Dataset**: CIFAR-100

**Network:** CNN-13

**Number of labeled samples:** 4000, 10000

# Partial Model

The first ablation experiment was conducted on variants of RLGSSL which removed certain parts of the model:

- **without RL loss**
- **without supervised loss**
- **without consistency loss**
- **without EMA:** teacher model is removed completely
- **without mixup:** mixup operation is removed completely

# Partial Model

| | RLGSSL | $-$w/o $\mathcal{L}_{\text{rl}}$ | $-$w/o $\mathcal{L}_{\text{sup}}$ | $-$w/o $\mathcal{L}_{\text{cons}}$ | $-$w/o EMA | $-$w/o mixup |
|---|---|---|---|---|---|---|
| CIFAR-100/4000 | $\mathbf{36.92}_{(0.45)}$ | $44.92_{(0.55)}$ | $39.52_{(0.58)}$ | $38.78_{(0.48)}$ | $43.12_{(0.52)}$ | $40.12_{(0.51)}$ |
| CIFAR-100/10000 | $\mathbf{29.12}_{(0.20)}$ | $33.12_{(0.52)}$ | $32.67_{(0.45)}$ | $31.48_{(0.32)}$ | $32.84_{(0.45)}$ | $31.48_{(0.32)}$ |

Results of different partial models vs the full model

# Reward Function

The second experiment revolved around tweaking different parameters for the reward function:

- **constant reward** $\mathcal{R} = 1$
- **using only labeled data for reward** $\mathcal{R} : \mu = 0$
- **replace MSE with KL divergence**
- **replace MSE with JS divergence**
- **differentiable reward function**

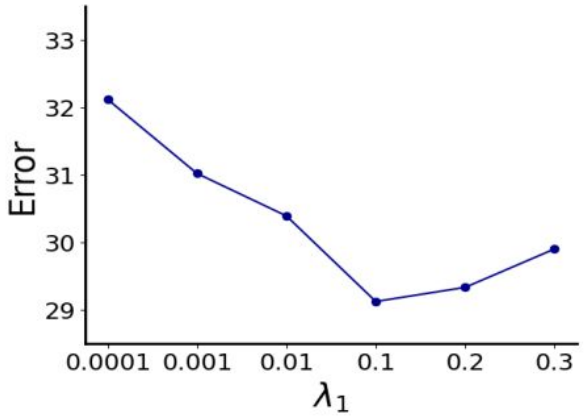
# Reward Function

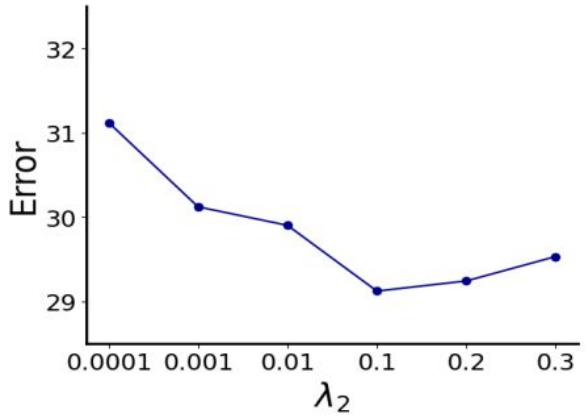
| | RLGSSL | $\mathcal{R}=1$ | $\mathcal{R}:\mu=0$ | $\mathcal{R}(\text{MSE}\rightarrow\text{KL})$ | $\mathcal{R}(\text{MSE}\rightarrow\text{JS})$ | $\mathcal{R}:\text{w/o sg}[\theta]$ |
|---|---|---|---|---|---|---|
| CIFAR-100/4000 | $\mathbf{36.92}_{(0.45)}$ | $39.52_{(0.63)}$ | $39.54_{(0.33)}$ | $38.02_{(0.42)}$ | $39.52_{(0.45)}$ | $40.62_{(0.55)}$ |
| CIFAR-100/10000 | $\mathbf{29.12}_{(0.20)}$ | $31.25_{(0.62)}$ | $32.37_{(0.57)}$ | $31.12_{(0.52)}$ | $31.39_{(0.68)}$ | $32.12_{(0.62)}$ |

Results of different reward functions vs the standard model

# Hyperparameter Analysis

The trade-off parameters for the supervised loss and consistency loss were analysed by running multiple models with different ranges of values for $\lambda_1$ and $\lambda_2$.



# BONUS: Reward Weighting

# Reward Weighting

In the paper, the authors clearly state that the KL-divergence weighting of the reward *encourages the predictions to exhibit greater discrimination*; this suggests the focus on **exploitation**.

It is well known that SSL has the **confirmation bias** problem, which can be directly associated with **exploration** in Reinforcement Learning.

# Reward Weighting

Our suggestion is a reformulation of the RL loss used in the paper that will focus in the early stages of learning also on **exploration.**

More specifically, we propose adding an **annealed prediction-entropy bonus** to the reward weighting term, with the entropy coefficient decaying from 0.5 to 0 during training, encouraging exploration early on and confident pseudo-label exploitation later.

$$\mathcal{L}_{rl}(\theta) = -\mathbb{E}_{x_i^u \in \mathcal{D}^u}(\mathrm{KL}(e, P_\theta(x_i^u)) + \beta H(P_\theta(x_i^u)))\mathcal{R}(s, a; sg[\theta])$$

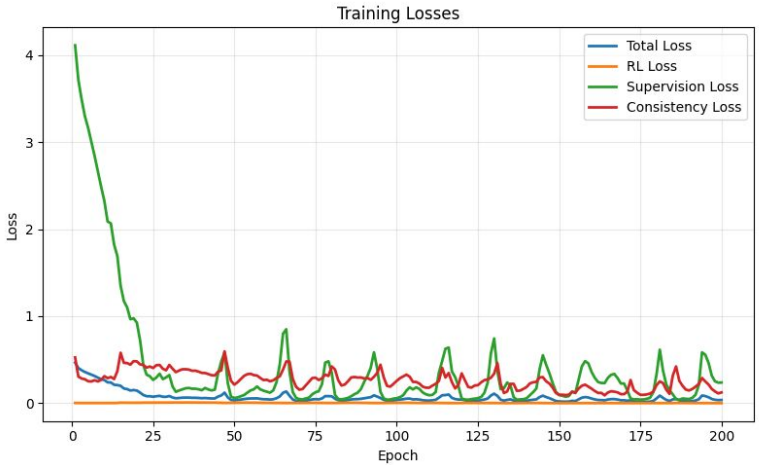
# Reward Weighting Study

| Dataset | RLGSSL | RLGSSL-Plus |
|---|---|---|
| CIFAR-100/10000 | 83.35% | 84.90% |

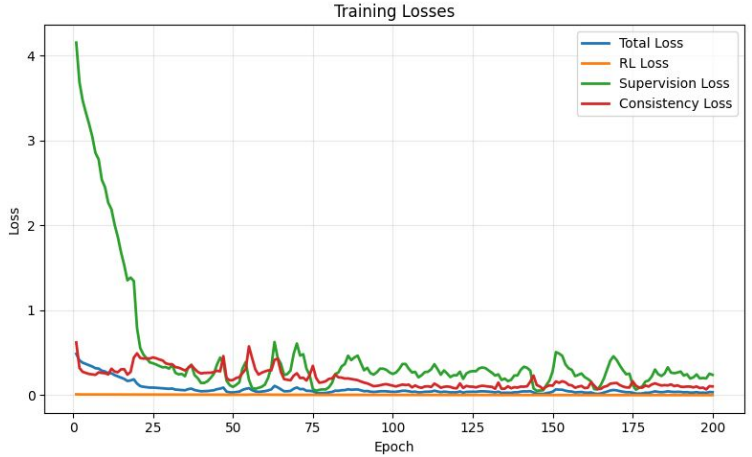
Table 2.1: Test error (%) on CIFAR-100 with 10,000 labeled samples.

Source code: https://github.com/VladWero08/one-arm-bandit-ssl

# Reward Weighting Study




RLGSSL Loss




RLGSS-Plus Loss

Source code: https://github.com/VladWero08/one-arm-bandit-ssl

# Thank you!